

Two-Week Team Evaluation Task

Researcher Data Validation, Customer Discovery, Prototype, and Pitch

This document outlines the rules, task expectations, role responsibilities, customer interview requirements, deliverables, submission instructions, and evaluation rubric for the Scholarix AI team-based selection round.

Duration	2 weeks
Team format	2 people: 1 Product & Research Operations candidate + 1 Full Stack Developer candidate
Input data	Researcher profile, publications, and broad-impact/search-result files for approximately 50 authors
Customer discovery	Each team must interview 2 potential users and connect those findings to the product direction
Goal	Interpret messy research data, validate it, choose a product direction, build a focused prototype, and pitch it
Constraint	No paid APIs, paid tools, paid credits, or services requiring payment
Final deadline	All code and supporting files must be submitted by Friday, August 28, 2026.

1. Purpose of the Evaluation

Scholarix AI is evaluating candidates not only for individual skill, but for how they collaborate in a realistic product-and-engineering setting. The project involves ambiguous data, incomplete information, customer/user interpretation, technical validation, and early-stage product choices.

The goal is not to produce a perfect production system. The goal is to demonstrate judgment: how the team understands users, questions data quality, verifies information, uses APIs responsibly, builds something useful, communicates uncertainty, and explains tradeoffs.

2. Team Structure and Role Responsibilities

Each team must include exactly two members: one Product & Research Operations candidate and one Full Stack Developer candidate. Teams should divide responsibilities clearly, but both members should understand and be able to explain the final solution.

The final submission should make it clear what the Product candidate contributed, what the Full Stack candidate contributed, and how both roles influenced the final product direction.

Product & Research Operations Candidate: What You Own

- Own the customer discovery process, including identifying 2 potential users, conducting interviews, summarizing findings, and explaining why those users are relevant.
- Define the user problem: who the user is, what issue they face, why the provided data matters to them, and what would make them use the product.
- Design and justify the customer interview questions. Do not only list questions; explain why each question was chosen and what you expected to learn from it.
- Translate customer findings into product requirements, user stories, acceptance criteria, and feature priorities.
- Explain how messy, incorrect, duplicated, or incomplete data affects user trust and product value.
- Work with the developer to decide what should be built based on the user interviews and data investigation, not just personal assumptions.
- Lead the product brief and final pitch narrative: problem, users, chosen direction, why it matters, tradeoffs, and next steps.

Full Stack Developer Candidate: What You Own

- Own the technical exploration of the provided files and identify what can be validated, enriched, corrected, visualized, or exposed through a prototype.
- Investigate appropriate free/public APIs or open-source tools, such as OpenAlex, Crossref, Semantic Scholar, ORCID, DBLP, arXiv, PubMed, or other relevant public sources.
- Build a focused proof of concept based on the product direction chosen from the customer interviews and data analysis.
- Design the technical workflow, such as data cleaning, validation, correction suggestions, confidence scoring, source comparison, search/filtering, API design, UI, dashboard, network visualization, or review queue.
- Document how the system handles incorrect, missing, duplicated, outdated, or uncertain data.
- Provide clear setup/run instructions, assumptions, limitations, and basic tests or validation examples where appropriate.
- Be prepared to explain technical tradeoffs: what you automated, what still requires human review, where the data/API failed, and what you would build next.

3. Data Provided

Each team will receive files for approximately 50 authors. The files are intentionally treated as imperfect input data. Some fields may be missing, incorrect, duplicated, outdated, inconsistent, or difficult to verify.

Teams should not assume the provided data is correct. They should investigate it, identify weaknesses, and decide what product direction best addresses the user problem.

File type	Typical contents	Examples of issues teams may detect
profile.json	Author-level metadata such as ID, name, affiliation, topics, keywords, metrics, yearly counts, verification fields, and ORCID	Outdated affiliations, ambiguous ORCID/institution records, inflated or inconsistent metrics, unclear topic labels,

	information.	missing profile links.
publications.json	Publication-level records such as DOI, title, journal, year, citation count, authors, sources, citations by source, and deep-verification fields.	DOI/title mismatch, author disambiguation problems, citation disagreement across sources, false negatives in verification, missing open-access PDFs, duplicate publications.
broad_impact.json	Search or broad-impact style results such as URL, snippet, category, relevance score, and reasoning.	Incorrect category assignment, weak relevance reasoning, duplicate sources, news vs publication confusion, hallucinated or low-quality relevance explanations.

4. Exact Task

Using the provided author files, two customer/user perspectives, and your own interviews with 2 potential users, your team must choose one focused product direction and build a small proof of concept that makes the data more useful, trustworthy, explainable, or actionable.

You may choose your own direction. We intentionally do not want to prescribe the solution, because the evaluation is designed to reveal how your team interprets messy data, customer needs, technical constraints, and product opportunities.

Your final solution must clearly show how customer discovery influenced what you built. For example, if users say they do not trust citation counts, your prototype might focus on source comparison and confidence labels. If users struggle to find collaborators, your prototype might focus on search, filtering, and networks. If users worry about incorrect profiles, your prototype might focus on correction workflows or admin review queues.

Possible product directions

- A correction or validation system that flags likely incorrect records and proposes fixes.
- A confidence-scoring workflow that explains why records are trusted, uncertain, or suspicious.
- A researcher discovery UI that helps a user search, filter, and understand researcher profiles.
- A duplicate-detection tool for authors or publications.
- A source comparison dashboard showing agreement/disagreement across APIs or datasets.
- A publication verification workflow that checks DOI, title, author, venue, year, and citation metadata.
- A co-author, topic, or publication network visualization.
- An admin review queue for records requiring human review.
- A lightweight API-backed enrichment pipeline that improves missing or stale records.

You do not need to build all of these. A focused, well-explained solution is better than a broad, unfinished system.

5. Customer Discovery Requirement

Each team must interview 2 potential users. These interviews are a required part of the task, not an optional activity.

The users do not need to be perfect enterprise customers. They should be realistic people who may care about researcher discovery, publication trust, faculty/lab search, collaboration mapping, research analytics, funding strategy, or data correction. Examples include graduate students, research assistants, faculty members, lab managers, research administrators, librarians, academic program staff, startup founders in research tooling, or people who frequently search for researchers/papers.

For each interview, your team should understand what issue the user faced, why the issue mattered, whether they would use a product like this, and what features would make it useful or trustworthy.

Required interview output	What to include
Who they are	Role, context, and why this person is relevant to Scholarix AI.
Issue they faced	A concrete problem they have experienced related to researcher discovery, publication search, profile trust, collaboration finding, research analytics, or messy academic data.
Why they would use this	What value the product would provide to them and what job they would hire it to do.
Features they want	Specific workflows, UI elements, validation features, search/filtering, confidence labels, networks, correction tools, exports, or explanations they requested or implied.
Trust concerns	What would make them distrust the data, such as stale affiliations, incorrect publications, unclear sources, duplicates, or unexplained recommendations.
Product implications	What your team decided to build or prioritize because of this interview.

Question design requirement	Explanation expected
Question asked	Write the actual interview question.
Why this question was chosen	Explain what uncertainty, assumption, user need, or product decision the question was meant to test.
What you learned	Summarize the user response and the insight gained.
How it affected the prototype	Explain what changed in your product direction, features, workflow, UI, or technical implementation because of the answer.

6. Provided Customer/User Perspectives

Each team will receive two written customer/user perspectives. Use them along with your own interviews to guide your product direction. For each user, explain who they are, why they matter, what they need, and what data-quality issues would affect their trust.

Question	Expected answer
Who is the user?	Identify the user type, role, and context.
Why are they relevant?	Explain why this user would care about researcher data, publications, discovery, verification, analytics, or collaboration patterns.
What job are they trying to do?	State the practical goal they want to accomplish.
What would make the product useful?	Describe features, workflows, or outputs that would help them.
What would make them lose trust?	Identify data errors, unclear sources, stale information, duplicates, or unexplained recommendations.

7. Data Validation Requirements

- Do not assume the provided data is true.
- Use free and public sources where appropriate to verify or enrich the data.
- Clearly distinguish verified facts, likely matches, uncertain matches, suspected errors, and unresolved issues.
- Track which sources were used for each verification decision.
- Explain confidence or uncertainty in a way a non-technical user can understand.
- If different sources disagree, document the disagreement instead of hiding it.
- If a record cannot be verified, mark it clearly rather than forcing a conclusion.
- Connect validation choices to user needs. For example, do not validate every field equally if your interviews show users care most about affiliation accuracy, publication relevance, or duplicate profiles.

Suggested free sources

- OpenAlex
- Crossref
- Semantic Scholar
- ORCID
- DBLP
- arXiv
- PubMed
- University profile pages
- Lab pages
- Personal faculty websites
- Public CVs or publication pages

You are not expected to use every source. Choose sources that fit your product direction and explain why.

8. Rules and Constraints

Rule	Requirement
No paid tools	Do not use paid APIs, paid credits, paid scraping services, or any service that

	requires payment.
Respect source policies	Do not aggressively scrape websites. Respect robots.txt, rate limits, and public-source terms. Prefer APIs when available.
Responsible AI use	AI tools may be used, but you must document how they were used and how outputs were checked.
No confidential data	Do not use confidential information from previous employers, labs, coursework, or private projects.
No production access	This task does not involve private Scholarix AI code, systems, credentials, or production data.
Scope discipline	Build a focused proof of concept. Do not attempt to build an entire platform.
Explain uncertainty	Do not present uncertain or inferred information as verified fact.
Reproducibility	The evaluator should be able to run or understand your work from the README and submitted materials.

9. Required Deliverables

Deliverable	What it should include
1. GitHub repository for code	All implementation work must be submitted through a GitHub repository. This includes source code, scripts, notebooks, frontend/backend code, API integrations, validation logic, tests, and any code needed to run the prototype. The repository should include meaningful commits and a clear README. If the repository is private, make sure access is granted for review.
2. README in GitHub repository	Setup steps, how to run the project, dependencies, expected inputs/outputs, known limitations, API/source usage, and a short explanation of the repository structure. The README should make it possible for a reviewer to understand and run the prototype without additional clarification.
3. Product brief	User/customer analysis, chosen problem, product direction, feature prioritization, and why this direction matters.
4. Customer interview report	2 potential users interviewed, who they are, why they are relevant, the issue they faced, why they would use this product, and what features they want.
5. Interview question rationale	The questions asked, why each question was chosen, what the team learned, and how those answers shaped the product/prototype.
6. Data investigation report	Examples of verified records, questionable records, likely errors, duplicate candidates, source disagreements, or missing data.
7. Prototype/demo	Working proof of concept: UI, dashboard, network, CLI, notebook, API, or report generator.
8. Final pitch deck or short slide document	10-minute presentation/demo covering users, problem, data issues, chosen direction, prototype, tradeoffs, and next steps.
9. AI/tool usage note	What AI tools were used, for what purpose, and how generated output was validated.
10. Google Drive submission folder	All non-code deliverables should be submitted as relevant files in the provided Google Drive folder. This includes the product brief, customer interview report, interview question rationale, data validation report, final pitch deck/slides, demo recording or screenshots, output samples, AI/tool usage

10. Prototype Minimum Expectations

The prototype can be simple, but it must demonstrate a clear idea and operate on the provided data. It should not be only a static slide deck or only a written report.

The prototype must also reflect customer discovery. Your team should be able to point to at least 2-3 features, workflows, UI decisions, or validation choices and explain which interview insight influenced them.

At minimum, the prototype should show one or more of the following:

- Incorrect or suspicious records being flagged.
- Duplicate authors or duplicate publications being identified.
- Missing information being enriched from a public source.
- Multiple sources being compared for agreement/disagreement.
- Confidence scores or labels being assigned and explained.
- Suggested corrections being shown to a user or admin reviewer.
- Researchers, publications, topics, or co-authors being connected in a useful view.
- A search/filter/exploration experience that helps a user understand and trust the data.
- A feature or workflow directly motivated by the 2 user interviews.

11. Final Presentation and Interview Structure

Each team selected for the final round will have a 30-minute final review slot: 5 minutes for team introductions, 10 minutes for the presentation/demo, 5 minutes for Q&A, and 10 minutes for interview/discussion.

Only teams whose full task submission is completed by the Friday, August 28, 2026 deadline will remain eligible to be called for the final interview round. Incomplete or late submissions will not move forward.

The list of teams/candidates called for the final interview will be shared by Sunday, August 30, 2026.

Final interviews will be scheduled from Monday, August 31 to Friday, September 4, 2026, mostly in the evenings.

During the presentation and interview discussion, teams should be prepared to answer questions such as:

1. Who are the two users you interviewed, and why are they relevant?
2. What issue did each user face?
3. Why would those users use a product like this?
4. What features or workflows did the users want?
5. Why did you choose your interview questions?
6. What did you learn from the interviews?

7. What problem did your team choose to solve?
8. What did you discover about the provided data?
9. What sources/APIs did you use and why?
10. What did your prototype do?
11. What did you build specifically because of customer discovery?
12. What worked well?
13. What did not work or remained uncertain?
14. What would you build next?

12. Evaluation Rubric

Category	Points	What we are looking for
Product judgment	15	Clear user problem, meaningful product direction, thoughtful prioritization, practical workflow.
Customer discovery execution	15	2 real/reasonable potential users interviewed, clear issue identification, useful feature insights, and strong explanation of why questions were chosen.
Customer-to-product translation	10	Clear evidence that interview findings shaped product requirements, feature choices, prototype behavior, UI, or validation logic.
Data judgment	15	Ability to detect incorrect, duplicate, incomplete, stale, or suspicious data; honest uncertainty handling.
API/source investigation	15	Appropriate use of free public sources/APIs; clear source comparison and limitations.
Prototype/demo	15	A focused working proof of concept that uses the provided data and demonstrates the chosen idea.
Technical clarity/testing	7	Readable code, setup documentation, basic test cases or validation examples, clear limitations.
Collaboration	5	Clear role division and evidence that product and engineering informed each other.
Pitch quality	3	Clear story, concise demo, honest tradeoffs, practical next steps.

13. What We Are Not Looking For

- A perfect production-ready system.
- Paid API usage or paid tooling.
- Blind trust in the provided data.
- Interview questions listed without explaining why they were asked.
- A prototype that is disconnected from customer discovery.
- A broad, unfocused mega-dashboard that does not solve a clear user problem.
- A fancy UI with weak reasoning.
- Large-scale scraping or source-policy violations.
- AI-generated claims without verification.
- A solution that hides uncertainty instead of surfacing it.

14. What We Are Looking For

- A focused product direction chosen from user needs and data reality.
- Clear evidence that the team interviewed 2 potential users and used those insights.
- A thoughtful approach to verifying and correcting messy research data.
- Clear explanation of confidence, uncertainty, and source disagreement.
- A small but functional prototype that demonstrates the idea.
- Good collaboration between product and engineering.
- Practical tradeoffs and honest limitations.
- A clear pitch explaining why the solution matters for Scholarix AI.

15. Submission Checklist

- Team names, roles, and emails included.
- GitHub repository link submitted.
- Google Drive team folder created and all non-code deliverables uploaded.
- README includes setup and run instructions.
- Product brief included.
- 2-user customer interview report included.
- Interview questions, question rationale, and findings included.
- Explanation of what was developed based on customer discovery included.
- Data investigation/validation report included.
- Prototype/demo link or screen recording included.
- Final pitch deck or slide document included.
- AI/tool usage note included.
- All materials submitted by Friday, August 28, 2026.
- Full submission completed by the deadline to remain eligible for the final interview.

16. How to Submit

Final deadline: Friday, August 28, 2026.

Only teams whose full task submission is completed by the deadline will remain eligible to be called for the final interview. Incomplete or late submissions will not move forward.

Each team must submit both a GitHub repository and a Google Drive folder submission. The GitHub repository should contain the code. The Google Drive folder should contain the rest of the task materials in a format that is easy to review.

GitHub submission

- Create one GitHub repository for your team code. Do not submit separate unrelated repositories unless clearly explained.

- Include all source code, scripts, notebooks, frontend/backend code, API integrations, validation logic, tests, and prototype assets required to run or understand the implementation.
- Include a README with setup instructions, dependencies, how to run the project, expected inputs and outputs, API/source usage, known limitations, and a short explanation of the repository structure.
- Use meaningful commits where possible so we can understand how the work developed over time.
- If the repository is private, make sure review access is granted before the deadline.

Google Drive submission

Upload all non-code deliverables to the following Google Drive folder:

<https://drive.google.com/drive/folders/1VQvQyn9F5X0Yix0DEIA1xSchL2nvz-Dx?usp=sharing>

Create one folder per team using this naming format:

Team_<ProductCandidateName>_<FullStackCandidateName>.

- Product brief: user problem, target users, chosen product direction, feature prioritization, user stories, acceptance criteria, and tradeoffs.
- Customer interview report: two potential users interviewed, who they are, why they are relevant, what issue they faced, why they would use this product, and what features they want.
- Interview question rationale: the exact questions asked, why each question was chosen, what assumption or product decision it tested, what was learned, and how it influenced the final prototype.
- Data validation report: issues found in the provided data, sources/APIs used for verification, confidence scoring or flagging logic, unresolved uncertainty, and examples of corrected or suspicious records.
- Prototype/demo materials: demo video, screenshots, deployed link if applicable, or a short walkthrough showing the product workflow and the data-validation behavior.
- Final pitch deck or slide document: 10-minute presentation/demo covering users, problem, data issues, chosen direction, prototype, tradeoffs, and next steps.
- Output samples: any generated CSV, JSON, annotated records, screenshots, diagrams, or sample API responses that help explain the result.
- AI/tool usage note: what AI tools were used, where they helped, and how outputs were verified.
- Optional supporting files: architecture diagram, API notes, source comparison notes, testing notes, or other material that makes the team's work easier to evaluate.

Submission expectation

- The GitHub repository link should be clearly included in the Google Drive submission, preferably in a short overview document or README PDF/Doc.
- Do not submit only a slide deck or only a GitHub repository. The submission should include both code and the product/research documentation needed to evaluate the full task.
- File names should be clear and reviewer-friendly. Avoid generic names such as final.pdf, doc1, or presentation_latest_v3.
- All final materials must be submitted by Friday, August 28, 2026.